# POC-03

## *Embedded Plant Monitoring System*

Casey Stijlaart

# Table of Contents

# 1. Introduction

This third iteration of my proof of concept demonstrates the development and validation of an embedded AI-based plant health monitoring system.

The system is designed to operate on low-cost hardware and provide real-time insights into plant conditions using sensor data and on-device machine learning (TinyML).

The primary goal of this POC is to verify that a trained machine learning model behaves consistently when deployed on an embedded device, compared to offline execution in a Python environment. Besides that, another goal was to implement further functionality, such as a web interface for user interaction and persistent logging of plant health data.

# 2. Core Changes

Compared to previous iterations, the following core changes were made:

- **Data Logging:** All collected data and inference results are now stored in CSV format on the ESP32's LittleFS filesystem. This allows for offline validation and analysis of the system's performance.
- **Web Interface:** A local web server is hosted on the ESP32, providing a user-friendly interface to view current plant settings, update preferences, and access stored logs. This enhances user interaction and accessibility of the system.
- **Model Validation:** A comprehensive validation process was conducted to ensure that the TinyML model deployed on the ESP32 produces identical predictions to the original Python implementation when given the same input features. This confirms the consistency and correctness of the model deployment.
- **Personalization:** Users can now define per-plant preferences for soil moisture, temperature, humidity, and light. These preferences influence the stress calculations and recommendations, allowing the system to adapt to different plant types without retraining the model.
- **Time Synchronization:** The system now uses NTP to synchronize with real-world time, ensuring accurate timestamps for data logging and user interaction. The timezone is set to Europe/Amsterdam (CET/CEST), and the system automatically adjusts for daylight saving time. If NTP synchronization fails, the system falls back to using internal timestamps.

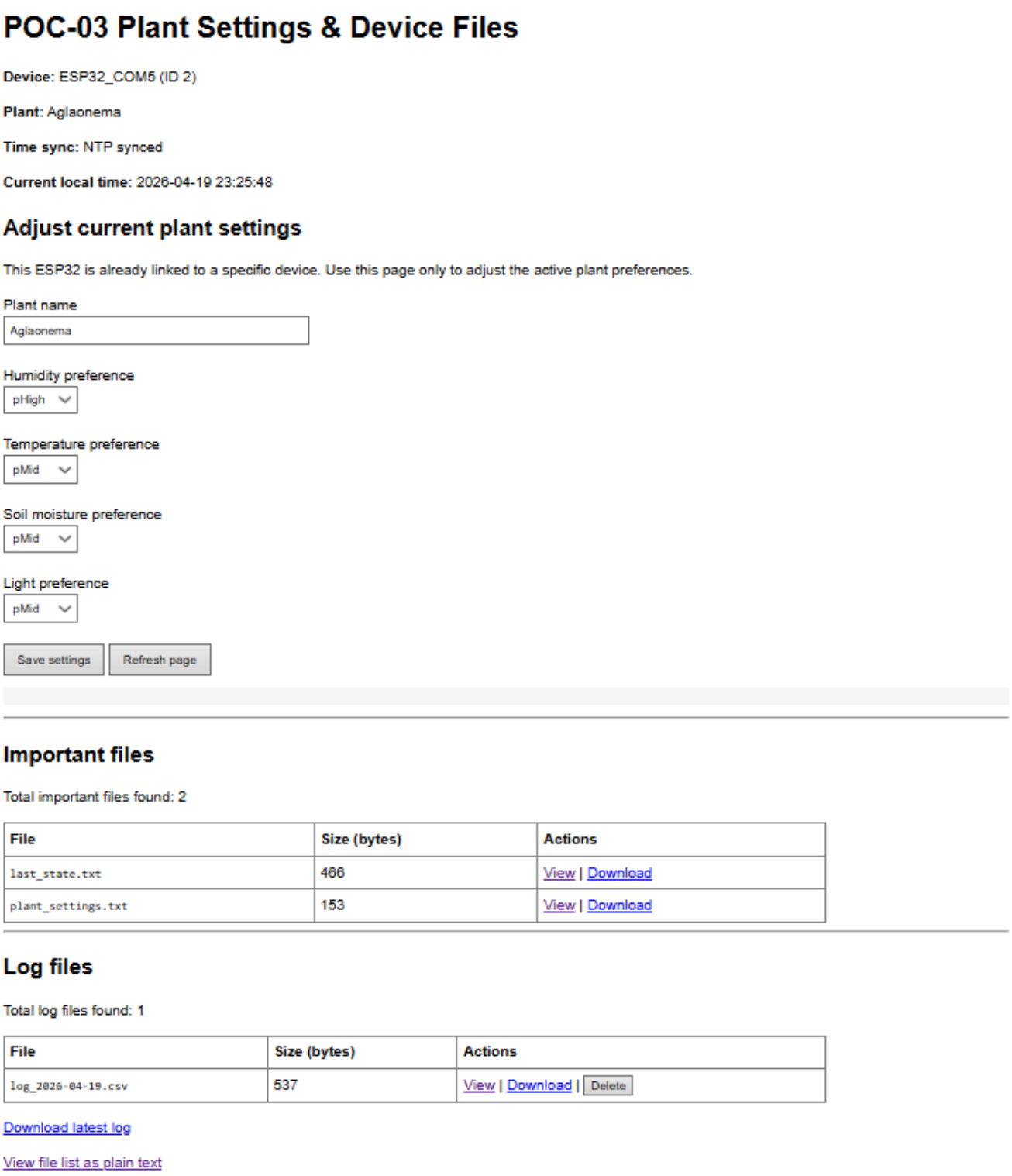
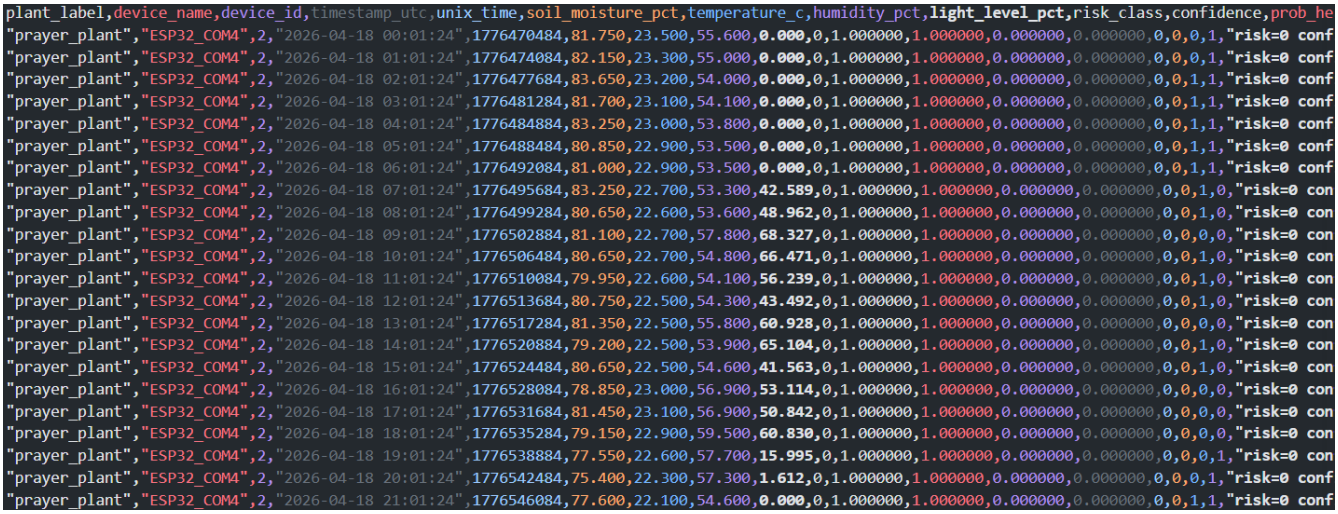



*Figure 1. Core Change : Web Interface via ESP32 Web Server*



```
plant_label,device_name,device_id,timestamp_utc,unix_time,soil_moisture_pct,temperature_c,humidity_pct,light_level_pct,risk_class,confidence,prob_he
"prayer_plant","ESP32_COM4",2,"2026-04-18 00:01:24",1776470484,81.750,23.500,55.600,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,0,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 01:01:24",1776474084,82.150,23.300,55.000,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,0,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 02:01:24",1776477684,83.650,23.200,54.000,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,1,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 03:01:24",1776481284,81.700,23.100,54.100,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,1,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 04:01:24",1776484884,83.250,23.000,53.800,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,1,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 05:01:24",1776488484,80.850,22.900,53.500,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,1,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 06:01:24",1776492084,81.000,22.900,53.500,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,1,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 07:01:24",1776495684,83.250,22.700,53.300,42.589,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 08:01:24",1776499284,80.650,22.600,53.600,48.962,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 09:01:24",1776502884,81.100,22.700,57.800,68.327,0,1.000000,1.000000,0.000000,0.000000,0,0,0,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 10:01:24",1776506484,80.650,22.700,54.800,66.471,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 11:01:24",1776510084,79.950,22.600,54.100,56.239,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 12:01:24",1776513684,80.750,22.500,54.300,43.492,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 13:01:24",1776517284,81.350,22.500,55.800,60.928,0,1.000000,1.000000,0.000000,0.000000,0,0,0,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 14:01:24",1776520884,79.200,22.500,53.900,65.104,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 15:01:24",1776524484,80.650,22.500,54.600,41.563,0,1.000000,1.000000,0.000000,0.000000,0,0,1,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 16:01:24",1776528084,78.850,23.000,56.900,53.114,0,1.000000,1.000000,0.000000,0.000000,0,0,0,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 17:01:24",1776531684,81.450,23.100,56.900,50.842,0,1.000000,1.000000,0.000000,0.000000,0,0,0,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 18:01:24",1776535284,79.150,22.900,59.500,60.830,0,1.000000,1.000000,0.000000,0.000000,0,0,0,0,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 19:01:24",1776538884,77.550,22.600,57.700,15.995,0,1.000000,1.000000,0.000000,0.000000,0,0,0,1,"risk=0 con
"prayer_plant","ESP32_COM4",2,"2026-04-18 20:01:24",1776542484,75.400,22.300,57.300,1.612,0,1.000000,1.000000,0.000000,0.000000,0,0,0,1,"risk=0 conf
"prayer_plant","ESP32_COM4",2,"2026-04-18 21:01:24",1776546084,77.600,22.100,54.600,0.000,0,1.000000,1.000000,0.000000,0.000000,0,0,1,1,"risk=0 conf
```

*Figure 2. Core Change : Data Logging in CSV Format*

# 3. System Overview

The system consists of an embedded device (ESP32) equipped with multiple environmental sensors. These sensors measure key variables that influence plant health:

- Soil moisture
- Ambient temperature
- Relative humidity
- Light intensity

The collected data is processed locally on the ESP32, where a TinyML model performs inference to classify plant health status and generate recommendations.

The system logs all collected data and inference results to a local CSV file stored on the device.This allows for offline validation and analysis.

The following section describes how these components interact during system operation.

# 4. System Operation

The system operates in a continuous loop consisting of four main steps:

1. **Data Acquisition:** Sensor readings are collected at regular intervals.
2. **Data Structuring:** The raw sensor values are combined into a structured data snapshot, including timestamps and derived features.
3. **Model Inference:** The TinyML model processes the input data and outputs:
    - A predicted risk class (e.g., Healthy, Moderate Stress, High Stress)
    - Confidence scores for each class
4. **Recommendation Generation:** Based on the predicted class, the system generates actionable recommendations, such as:

    - Water the plant
    - Increase humidity
    - Adjust temperature
    - Modify light exposure

All outputs are logged for further analysis.

# 5. System Architecture

The system follows a fully local architecture, ensuring independence from external services:

- Data is processed entirely on-device
- No cloud connectivity is required
- Logs are stored locally and can be retrieved via a lightweight HTTP server

This design supports:

- Low-cost deployment
- Offline functionality
- Scalability across multiple devices

The overall system architecture is illustrated in the corresponding diagram.

# 6. Model Validation

To ensure the reliability of the system, a validation process was performed comparing on-device inference with offline Python-based inference.

## 6.1. Methodology

The validation consisted of the following steps:

1. Data was collected from the ESP32 during runtime and stored as a CSV log file.
2. The same dataset was used as input for a Python script running the exported model.
3. Predictions from both environments were compared.

## 6.2. Results

The results showed that:

- The predicted risk classes were identical across both environments.
- Confidence values matched within floating-point precision.
- No discrepancies were observed in model behavior.

This confirms that the model behaves deterministically across deployment environments, ensuring consistency between development and production.

[comparison] | */comparison.png*

*Figure 3. Comparison of Predictions*

## 6.3. Validation Scope and Limitations

While the validation confirms consistency between environments, it is important to note its scope:

- The validation used data generated by the deployed system itself.
- External datasets or edge-case scenarios were not included.
- Long-term environmental variability was not explicitly tested.

Therefore, while the results confirm correctness under normal conditions, broader validation may be required for full robustness.

# 7. Future Improvements

- Improve ML model with temporal features
- Add predication for a time window, e.g. "in 1 hour" water or "in 2 hours" increase light.
- Revise best solution for data storage which is lightweight and (in)directly accessible for users, e.g. via web interface or application.
- Optimize power usage for standalone deployment
- Give better suggestion for user actions, such as the soil has been too moist for a long time, so watering is not recommended, or the plant has been in high stress for a long time, so more drastic actions are needed.
- A solution for remote access, e.g. via MQTT or cloud integration, while maintaining security and privacy.
- An approach that allows for multiple plants to be monitored trough a singular interface, e.g. a mobile app or web dashboard, while keeping the system scalable and user-friendly.

# 8. Conclusion

This proof of concept successfully demonstrates a fully embedded AI-based plant health monitoring system using TinyML.

The validation confirms that the deployed model produces consistent and reliable predictions when compared to offline execution. This ensures that the system can be trusted to operate independently in real-world conditions.

The results align with the original goal of creating an accessible, low-cost, intelligent plant monitoring solution for hobbyist users, while providing a strong foundation for further development and refinement.